

Team SimGym

Track 4 : Ai Customer Support Agent for Commerce

1. Introduction

This technical document explains how the AI-Based E-Commerce Storefront with Shopping Assistant Chatbot was built, how different components are connected, what role AI plays in the system, how failures are handled, and the limitations of the current implementation.

The main objective of this project is to improve the online shopping experience by combining product browsing and customer support into a single intelligent platform.

2. System Architecture

Main Components

The system is built using React and follows a component-based architecture.

Core Components

1. App Component

This is the main parent component that controls the entire application. It manages:

- User login and logout
- Cart items
- Wishlist items
- Product data flow
- Communication between Storefront and ChatBot

2. Storefront Component

This component displays all available products and handles:

- Product listing
- Add to Cart functionality
- Wishlist management
- User profile display
- Cart count display

3. ChatBot Component

This is the AI-based support system that helps users with:

- Product recommendations
- Cart summary
- Order tracking
- Return initiation
- Shipping details
- Payment-related queries
- Shopping suggestions

4. Chat Widgets

Special widgets are used for structured responses such as:

- ProductWidget
- OrderTrackingWidget
- ReturnWidget
- CartSummaryWidget

These improve user understanding instead of only showing text replies.

5. Mock AI Engine

The chatbot currently uses `simulateAIResponse()` from the mock AI utility instead of a real AI API.

This function processes user queries and generates predefined intelligent responses.

3. Data Flow and Component Connection

Step-by-Step Data Flow

1. User opens the application
2. App component loads product data and stored user profile
3. Storefront displays products to the user
4. ChatBot receives product data, cart data, and user information
5. User sends a message to the chatbot

6. ChatBot sends input to `simulateAIResponse()`
7. AI response is generated
8. Response is displayed as text or widget
9. If recommended, product can be directly added to cart from chatbot

This creates a connected shopping and support experience.

4. Key Implementation Decisions and Why

Decision 1: React Frontend Architecture

Reason: React allows reusable components, better state management, and clean UI development. It also improves scalability for future upgrades.

Decision 2: Local Storage for User Login

Reason: Instead of full backend authentication, local storage was used to save user profile data for prototype simplicity and faster implementation.

Decision 3: Mock AI Instead of Live API

Reason: Real AI APIs require API keys, backend security, internet dependency, and additional cost. For project demonstration, simulated AI responses were more practical.

Decision 4: Widget-Based Chat Responses

Reason: Structured widgets improve readability and make chatbot responses more useful than plain text messages.

5. AI/LLM Role vs Deterministic Code

What AI Handles

The chatbot handles user interaction where natural language is useful, such as:

- Product recommendations
- Shopping guidance
- Return policy explanation
- Order tracking assistance
- Cart summary explanation
- Customer support conversations

AI improves flexibility in handling user questions.

What Deterministic Code Handles

Fixed logic is handled by standard code, such as:

- Login/logout process
- Cart quantity updates
- Wishlist add/remove
- Product listing
- User profile storage
- Button actions
- Widget rendering

Why This Separation Was Chosen

AI should not control sensitive actions like cart calculation or user authentication because these require accuracy and reliability.

Deterministic code handles exact operations, while AI handles conversational support.

This improves both safety and user experience.

6. Failure Handling and System Degradation

Case 1: If AI Returns Invalid or Unexpected Response

Problem: The chatbot may return incorrect or incomplete output.

Handling: Fallback default responses are shown, and structured widgets are used only when response validation passes.

System Degradation: The chatbot still continues basic text support instead of breaking the UI.

Case 2: If User Input is Unexpected

Problem: Users may type unclear, empty, or unsupported questions.

Handling: Input validation removes empty messages and suggestion prompts help users ask valid questions.

System Degradation: The chatbot guides users using predefined suggestions.

Case 3: If Local Storage Fails

Problem: Stored user data may be corrupted.

Handling: Try-catch blocks safely handle JSON parsing errors and reset invalid data.

System Degradation: The user is treated as a new user instead of crashing the system.

Case 4: If Real API Integration Fails (Future Scope)

Problem: If future OpenAI or Shopify API becomes unavailable.

Handling: Fallback manual support responses and cached product information can be used.

System Degradation: Basic shopping features continue working even if AI support is limited.

7. Known Limitations

Current limitations include:

- No real payment gateway integration
- No live AI API integration
- No backend database for orders
- No real order management system
- No admin dashboard
- No merchant-side management panel
- No real shipping API integration
- Limited chatbot intelligence due to mock AI
- No mobile application version

These limitations exist because the project focuses on prototype-level product validation and core user experience.

8. Future Improvements

With more development time, the following improvements can be added:

- OpenAI GPT or Google Gemini API integration
- Secure backend with database support
- Real payment gateway integration
- Admin dashboard for merchants

- Order history and invoice management
- Real-time shipment tracking
- Push notifications
- Mobile app development
- Advanced recommendation engine using real AI models

These upgrades would move the project from prototype stage to production-ready platform.